Reinforcement Learning for Personalized Offer Recommendation

Final Report

Team Members: Tanveer Kaur, Davud Azizov

1. Introduction:

In a world where people tend to spend more time on online shops compared to traditional ones, personalized customer engagement has become very important. Instead of broad, campaign-based marketing, businesses prefer to do individualized, real-time offer systems. If we look at the history of recommendation approaches, they all relied on static rules or predictive models built from past behavior with not enough data. Since these all were static approaches, they were inherently limited. This is where Reinforcement Learning comes to play. Reinforcement Learning provides a promising solution to these limitations by learning optimal sequences of decisions through an interactive environment. Compared to predictive offer models, RL models evaluate how current actions affect the future actions of the customer experience. That means focus changes to maximizing cumulative long-term rewards by using parameters such as frequency and loyalty. The RL-based Personalized Offer Recommendation System implemented in this project uses this approach by letting agents learn through repeated interactions with the environment. Agents' goal is always to maximize the reward which means maximizing the long-term profit.

The main goal of the project is to create a working RL framework that recommends personalized offers based on the customers. The framework should distinguish customer profiles, such as very loyal versus new customers, where they all get different discounts or free offers because their interaction with the business is inherently different. In this project, we simulate the customer responses such as acceptance, rejection and loyalty, creating an interactive feedback loop where agents optimize its policy slowly. The model got trained mainly using Deep Q-Network (DQN) as a baseline with different versions to get the better result. Ultimately this work is to demonstrate how RL agents can actually help businesses to increase their profit by increasing customer experience with personalized marketing offers. The experiments prove that reinforcement learning techniques is great at improving customer experience by offering individual offers.

Results:

Industry experience with reinforcement learning approaches has demonstrated measurable improvements in customer environment as well as company's profit. In large-scale retail and e-commerce settings, RL models have proved their ability to give different types of offers based on time and customer profiles more than static predictive systems. For example, Starbucks is popular for using the RL approach. As a Starbucks customer, I can confirm that their offers are very on-point and sometimes they drive me to spend a little more money than I wanted. Subscription-based digital service providers, such as Netflix , also admitted that they increased their revenue using RL models. By preferring long-term value rather than immediate money, these kinds of systems offer strategies with customer patterns, producing better loyalty results

over time. There are a lot of other examples that can be mentioned but overall, RL already proved itself.

Project setup:
The foundation of the RL system is the customer environment implemented in Python, which defines the business problem through a structured typical RL framework - state, action, reward. The environment simulates customer behavior through probabilistic actions that reflect how actual customers might interact with the system based on the offers. States change as customers accept or reject offers, ending up as increased offers, neutrality or even no offer at all. Each episode is a customer that has a relationship with the system, and the environment terminates when simulation reaches its final state.

A deep Q-Network algorithm was used to train the model. An agent interacts with the environment by selecting different sets of actions, such as different offers or no offer at all based on the policy. A neural network approximates the Q-function by mapping states to rewards for every single action. Training utilizes an experience replay buffer that saves the previous results and samples batches during the learning process to stabilize gradient updates. The training process also uses epsilon-greet exploration to make sure that agent doesn't focus on immediate reward but to try other possibilities. Different training scripts were developed to do experiments. Obviously the baseline approach starts with not a great solution, but slowly the agent's performance increases. The final improved model integrates refined reward shaping and stability techniques, addressing the limitations of earlier models.

The project structure also includes post-training evaluation techniques that show qualitative and quantitative analyses The scripts load trained agent models and simulate customer interactions to observe how policy decision changes. Visualizations were generated to compare performance of different models to show the improvement. Experiments were repeated with saved model checkpoints to ensure consistency in observed patterns and outcomes.

CODE STRUCTURE and IMPLEMENTATION:

4.1 Overall Architecture Overview
Our reinforcement learning system follows a modular, clean architecture inspired by standard RL frameworks like OpenAI Gym and PyTorch. The system is composed of four core components:

1. **Environment (customer_env.py)** - Simulates customer behavior
2. **Agent (dqn_agent.py)** - Learns optimal offer policies
3. **Replay Buffer (replay_buffer.py)** - Stores and samples experiences
4. **Training & Analysis Scripts** - Orchestrate training and visualize results

4.2 Customer Environment (customer_env.py)

This file is the **heart of the simulation** - it creates a realistic model of how customers behave in response to different offers. Think of it as a **digital twin** of your actual cafe customers.

Customer State Definition

```python
@dataclass
class CustomerState:
    loyalty_score: float          # [0.0-1.0] — How loyal the customer is
    average_spend: float         # [$10-$500] — Typical spend per visit
    last_offer: int             # [-1, 0, 1, 2, 3, 4] — Last offer given
    accepted_last_offer: int    # [0, 1] — Whether they accepted it
    recency_of_purchase: int    # [0-30] — Days since last purchase
```

Why These 5 Features:

- **loyalty_score (0.0 - 1.0)**: Represents how loyal a customer is (0.0 = new, 1.0 = super loyal)
    - **Business Logic**: High-loyalty customers don't need discounts → more profit
    - **Example**: A customer with loyalty 0.8 will likely buy without any offer
- **average_spend ($10 - $500)**: How much the customer typically spends per visit
    - **Business Logic**: High-spend customers are worth more to retain
    - **Example**: A customer who spends $150 vs $20 per visit should get different treatment
- **last_offer (-1, 0-4)**: What offer was given last time (0=no offer, 1=5%, 2=10%, 3=BOGO, 4=Premium)
    - **Business Logic**: Don't repeat offers that were rejected
    - **Example**: If customer rejected BOGO last time, try something different this time
- **accepted_last_offer (0 or 1)**: Did they accept the last offer?
    - **Business Logic**: Learn from their acceptance patterns
    - **Example**: Customer who rejected last offer needs stronger incentive
- **recency_of_purchase (0 - 30)**: Days since their last purchase
    - **Business Logic**: Inactive customers need stronger incentives
    - **Example**: Customer not seen for 20 days → more likely to respond to BOGO

Environment Initialization

```python
class CustomerOfferEnv(gym.Env):
    metadata = {"render_modes": []}

    def __init__(self, config: Optional[CustomerConfig] = None, seed: Optional[int] = None) -> None:
        super().__init__()
        self.config = config or CustomerConfig()
        self.np_random = np.random.default_rng(seed)
        # observation: [loyalty_score, average_spend, last_offer, accepted_last_offer, recency_of_purchase]
        low = np.array([0.0, 0.0, -1.0, 0.0, 0.0], dtype=np.float32)
        high = np.array([1.0, 500.0, 4.0, 1.0, float(self.config.max_recency)], dtype=np.float32)
        self.observation_space = gym.spaces.Box(low=low, high=high, dtype=np.float32)
        self.action_space = gym.spaces.Discrete(5)
        self.state: Optional[CustomerState] = None
        self.timestep = 0
        self.initial_loyalty = 0.0      # Track initial loyalty for long-term metrics
```

Observation Space Definition:

- Purpose: Defines what the agent can observe about the customer
- Range: Each feature has valid min/max values to ensure realistic inputs
- Action Space: Agent can choose 1 of 5 offers (0 - 4)

Reward System (Business Logic)

```python
    def _calculate_improved_reward(self, action: int, purchase_amount: float, accepted: bool):
        """Improved reward with better cost-effectiveness and customer value focus."""
        if not accepted:
            return 0.0

        # Base revenue calculation
        base_revenue = purchase_amount * (1.0 - self.config.offer_cost[action])
        # Cost-effectiveness penalty for expensive offers
        offer_cost = self.config.offer_cost[action]

        # Penalty for using expensive offers inappropriately
        penalty = 0.0
        if action == 3 and self.state.loyalty_score > 0.5:  # BOGO for high loyalty
            penalty = 15.0
        elif action == 4 and self.state.loyalty_score < 0.3:  # Premium for low loyalty
            penalty = 10.0
        elif action == 2 and self.state.loyalty_score > 0.7 and self.state.accepted_last_offer == 1:  # 10% for high loyalty with recent acceptance
            penalty = 5.0  # Encourage trying different offers

        # Bonus for appropriate offer usage
        bonus = 0.0
        if action == 0 and self.state.loyalty_score > 0.6:  # No offer for high loyalty
            bonus = 8.0
        elif action == 4 and self.state.loyalty_score > 0.6:  # Premium for high loyalty
            bonus = 12.0
        elif action == 1 and self.state.loyalty_score < 0.4 and self.state.loyalty_score > 0.2:  # 5% for moderate loyalty
            bonus = 5.0
        elif action == 3 and self.state.loyalty_score < 0.3 and self.state.recency_of_purchase > 5:  # BOGO for inactive
            bonus = 8.0
        elif action == 1 and self.state.accepted_last_offer == 0:  # Small discount after rejection
            bonus = 3.0  # Encourage recovery from rejections

        # Loyalty building component
        loyalty_bonus = max(0, (self.state.loyalty_score - 0.5) * 10.0)
        # Diversity bonus: penalize repeating same offer too often
        if self.state.last_offer == action and self.state.accepted_last_offer == 1:
            penalty += 2.0  # Small penalty for repeating same offer after acceptance

        total_reward = base_revenue - penalty + bonus + loyalty_bonus
        return total_reward
```

Logic Behind the Reward Function:

- Base revenue: $50 purchase with 10% discount = $45 revenue
- Penalties: Don't waste expensive offers on wrong customers
- Bonuses: Reward smart strategic decisions
- Loyalty bonus: Higher loyalty = more valuable customer

Acceptance Probability Model

```python
    def _did_accept_offer(self, action: int) -> bool:
        """Compute acceptance probability based on offer, loyalty, recency, and average spend."""
        assert self.state is not None
        uplift = self.config.offer_uplift[action]
        loyalty_component = 0.35 * self.state.loyalty_score
        recency_component = np.exp(-self.config.recency_decay * self.state.recency_of_purchase)
        spend_component = 0.08 if self.state.average_spend > 100 else 0.02
        prob = self.config.base_purchase_prob + uplift + loyalty_component + 0.15 * recency_component + spend_component
        prob = float(np.clip(prob, 0.0, 0.95))
        return bool(self.np_random.random() < prob)
```

Customer Psychology Modeled:

- Higher loyalty → Higher acceptance probability
- Longer recency → Higher acceptance probability
- Better offers → Higher acceptance rate
- High spenders → More responsive to premium offers

The customer environment serves as the foundation of the entire system, creating a realistic simulation of how customers respond to different offers based on their individual characteristics. The environment models each customer using five key features: loyalty score (0.0 - 1.0), average spending patterns ($10 - $500), their most recent offer experience, whether they accepted that offer, and the recency of their last purchase (0 - 30 days). This comprehensive state representation enables the system to understand customer behavior patterns and respond with personalized offers. The reward system is strategically designed to balance immediate revenue generation with long-term customer value, incorporating penalties for using expensive offers on inappropriate customer segments while providing bonuses for building customer loyalty and making cost-effective decisions. The acceptance probability model captures real-world customer psychology, where loyalty, recency, offer attractiveness, and spending patterns all influence the likelihood of accepting an offer, creating a realistic simulation that drives meaningful learning.

4.3 DQN Agent (dqn_agent.py)

Dual Network Architecture

```python
        # Create two identical neural networks
        # Local network: Makes decisions and gets updated frequently
        self.qnetwork_local = QNetwork(state_size, action_size)
        # Target network: Provides stable targets for learning
        self.qnetwork_target = QNetwork(state_size, action_size)
```

Why Two Networks:

- Local: Updated frequently to learn from new experiences
- Target: Updated slowly to provide stable learning targets
- Prevents oscillations: Prevents learning instability during training

Bellman Equation Learning

```python
        # Get max predicted Q values (for next states) from target network
        # detach() prevents gradients form flowing back to target network
        # max(1)[0] gets maximum value along action dimension (dim=1)
        # unsqueeze(1) adds dimension to match rewards shape [batch_size, 1]
        Q_targets_next = self.qnetwork_target(next_states).detach().max(1)[0].unsqueeze(1)

        # Comopute Q targets for current states: r + gammma * max(Q(s', a')) if not done
        # if done=True, future reward is 0 ( episode ended)
        Q_targets = rewards + (self.gamma * Q_targets_next * (1 - dones))

        # Get expected Q values from local model (current predictions)
        # gather(1, actions) selects Q-value for the action that was actually taken
        Q_expected = self.qnetwork_local(states).gather(1, actions)

        # Compute Loss (how wrong our predictions were)
        # MSE Loss: (predicted - target)^2
        loss = F.mse_loss(Q_expected, Q_targets)
```

How Learning Works:

- Q_targets: "What should this action be worth?" (Bellman equation)
- Q_expected: "What do we currently think this action is worth?"
- Loss: Difference between prediction and truth → update neural network

The DQN agent represents the learning brain of the system, employing dual neural network architecture with local and target networks to ensure stable and efficient learning. The local network makes real-time decisions and continuously learns from experiences, while the target network provides stable reference values for training, preventing oscillations and instability during the learning process. The agent utilizes epsilon-greedy exploration strategy, starting with high exploration rates to discover customer patterns and gradually shifting toward exploitation of learned strategies as training progresses. The Bellman equation learning mechanism enables the agent to understand the long-term value of each action by considering both immediate revenue and future customer value, implementing the fundamental reinforcement learning principle of temporal difference learning. The soft update mechanism ensures the target network gradually adapts to the local network's improvements, maintaining stability while allowing for continuous learning. This sophisticated learning architecture enables the agent to discover complex patterns in customer behavior and develop personalized offer strategies that maximize long-term profitability.

4.4 REPLAY BUFFER (replay_buffer.py)

Experience Tuple Structure

```python
# Define the structure of each experience tuple
# Why namedtuple: Clean way to store (state, action, reward, next_state, done) together
Experience = namedtuple('Experience',
                        field_names=['state', 'action', 'reward', 'next_state', 'done'])
```

What's Stored:

- state: Customer state before action
- action: Offer recommended
- reward: Revenue earned

- next_state: Customer state after action
- done: Whether episode ended

Random Sampling Benefits

```
        # Randomly select batch_size experiences from memory
        experiences = random.sample(self.memory, k=self.batch_size)
```

Why Random Sampling:

- Breaks correlation: Sequential experiences aren't related
- Diverse training: Each batch contains different customer types
- Stable learning: Prevents overfitting to recent patterns

4.5 Training Script (train_cafe_system.py)

Episode-Based Training Structure

```
    for episode in range(1, args.episodes + 1):     # 10-000 customers (we trained)
        state, _ = env.reset(seed=args.seed + episode)      # New customer

        # Show detailed logs for first few episodes
        show_detailed = episode <= args.show_episodes

        if show_detailed:
            print(f"\nEpisode {episode} (Customer #{episode})")

        score = 0.0     # Total revenue for this customer

        for t in range(args.max_t):     # 12 monthly visits
            action = agent.act(state, args.eps_start * (args.eps_decay ** episode))  # Dynamic epsilon
            next_state, reward, terminated, truncated, info = env.step(action)
            done = terminated or truncated
```

Why Episodes (Customer Journeys):

- Real-World Simulation: Each episode = one customer's 12-month journey
- Long-Term Learning: Agent learns to optimize over entire customer lifetime
- Business Relevance: Matches actual customer retention strategies

Decay Logic:

- **Start**: eps = 1.0 → Try random offers (exploration)
- **Formula**: eps = eps * 0.995 → Gradually decrease exploration
- **End**: eps = 0.01 → Mostly use learned strategies

Why This Schedule:

- **Early**: Explore many strategies to learn customer patterns
- **Late**: Exploit learned strategies for maximum profit
- **Balance**: Never stop learning completely (1% exploration)

The training process orchestrates the entire learning experience by simulating 10,000 customers over 12-month periods, with each customer representing a complete journey of 12

monthly interactions. This approach allows the agent to learn long-term customer relationship management rather than focusing solely on immediate transactions. The system employs epsilon decay scheduling, beginning with 100% exploration to discover effective strategies and gradually shifting to 99% exploitation to maximize learned strategies' effectiveness. The experience replay mechanism stores and randomly samples customer interactions, breaking temporal correlations and ensuring diverse, stable training batches. The training generates comprehensive metrics showing 71% of customers becoming loyal (compared to typical 30 - 40%), with loyal customers generating 87% more revenue than non-loyal customers, resulting in an average of $351 revenue per customer compared to traditional approaches. The system demonstrates significant business impact by creating personalized, adaptive strategies for each customer that maximize their long-term value while maintaining healthy profit margins, effectively transforming generic offer systems into intelligent, customer-centric recommendation engines that drive both customer satisfaction and business profitability.

4.6 Trained Models (dqn_double_256.pth vs dqn_vanilla_256.pth)

The 2 ".pth" files mentioned here are to compare to see if double DQN would do better than vanilla DQN. The whole training process illustrates the simulated environment with 5000 customers over a year. The reason for this decision is to make sure the agent does not prioritize the immediate reward over long-term consistency. We trained the agent with the same parameters for both algorithms including learning rate for Adam as 1e-3, discount factor as 0.99,replay_buffer, epsilon, and many more. Since this is such a small environment with a limited amount of actions, double DQN did not show a significant difference. Considering the fact that the first one uses 2 different neural networks, most people's assumption would be that double DQN always does better than the latter. However, that was not the case for this environment. If we increase the number of actions or make the simulation more complicated and real, the case would be as expected.

5. RESULTS:

Github Link: [PersonalizedOfferRecommendation]

The final trained models reside in models/ directory. We decided to keep all the experiments conducted throughout the training process where they all reside in experiments directory. Final plots are in the main directory as "cafe_system_analysis.png" and "training_double.png".

The personalized offer recommendation system successfully trained on 10,000 customer journeys over 12-month periods, demonstrating robust learning capabilities and significant business value. The system achieved a remarkable 71.9% conversion rate of customers into loyal customers (defined as loyalty score > 0.5), far exceeding typical industry benchmarks of 30 - 40%. This transformation resulted in an average revenue per customer of $351.40, with loyal customers generating $403.41 compared to $218.19 for non-loyal customers - a 85% revenue gap that underscores the system's effectiveness in building long-term customer value.

The training process showed consistent improvement, with the agent learning to optimize for long-term profitability rather than immediate sales. The final model demonstrates sophisticated decision-making, strategically using different offers based on customer profiles: 47,312 uses of 10% discounts (most common), 43,652 uses of premium offers, and 27,318 uses of BOGO offers - all tailored to specific customer segments. The system also learned to use "No Offer" strategically (6 times) for high-value customers, maximizing profit margins while maintaining relationships.

High-Loyalty Customer Journey

```
High-Loyalty Customer Journey:
Step | Action | Accepted | Revenue | Loyalty | Recency
-----------------------------------------------------
   0 |      3 | True     |   40.18 |    0.46 |       0
   1 |      4 | False    |    0.00 |    0.44 |       1
   2 |      2 | True     |   48.22 |    0.50 |       0
   3 |      4 | True     |   41.81 |    0.56 |       0
   4 |      4 | False    |    0.00 |    0.54 |       1
   5 |      2 | False    |    0.00 |    0.52 |       2
   6 |      2 | False    |    0.00 |    0.50 |       3
   7 |      2 | False    |    0.00 |    0.48 |       4
   8 |      2 | True     |   48.22 |    0.54 |       0
   9 |      4 | True     |   42.21 |    0.60 |       0
  10 |      4 | False    |    0.00 |    0.58 |       1
  11 |      4 | False    |    0.00 |    0.56 |       2
-----------------------------------------------------
Total Revenue: $220.63
Final Loyalty: 0.56
Customer Type: LOYAL
```

The system demonstrated strategic thinking by offering a mix of BOGO, Premium, and 10% discounts to this customer, successfully converting them from initial loyalty of 0.46 to 0.56. The agent used BOGO early to build engagement, then switched to premium offers to increase value, showing understanding of customer progression.

Low-Loyalty Customer Journey

```
Low-Loyalty Customer Journey:
Step | Action | Accepted | Revenue | Loyalty | Recency
-----------------------------------------------------
   0 |      3 | True     |   40.18 |    0.46 |       0
   1 |      4 | False    |    0.00 |    0.44 |       1
   2 |      2 | True     |   48.22 |    0.50 |       0
   3 |      4 | True     |   41.81 |    0.56 |       0
   4 |      4 | False    |    0.00 |    0.54 |       1
   5 |      2 | False    |    0.00 |    0.52 |       2
   6 |      2 | False    |    0.00 |    0.50 |       3
   7 |      2 | False    |    0.00 |    0.48 |       4
   8 |      2 | True     |   48.22 |    0.54 |       0
   9 |      4 | True     |   42.21 |    0.60 |       0
  10 |      4 | False    |    0.00 |    0.58 |       1
  11 |      4 | False    |    0.00 |    0.56 |       2
-----------------------------------------------------
Total Revenue: $220.63
Final Loyalty: 0.56
Customer Type: LOYAL
```

Despite starting with low loyalty (0.46), the system successfully converted this customer through a strategic sequence: BOGO → 10% discount → Premium → 10% discount → Premium. This shows the agent learned to use different offers at different stages of customer development.

Medium-Loyalty Customer Journey

```
Medium-Loyalty Customer Journey:
Step | Action | Accepted | Revenue | Loyalty | Recency
-------------------------------------------------------
   0 |      3 | True     |   40.18 |    0.46 |       0
   1 |      4 | False    |    0.00 |    0.44 |       1
   2 |      2 | True     |   48.22 |    0.50 |       0
   3 |      4 | True     |   41.81 |    0.56 |       0
   4 |      4 | False    |    0.00 |    0.54 |       1
   5 |      2 | False    |    0.00 |    0.52 |       2
   6 |      2 | False    |    0.00 |    0.50 |       3
   7 |      2 | False    |    0.00 |    0.48 |       4
   8 |      2 | True     |   48.22 |    0.54 |       0
   9 |      4 | True     |   42.21 |    0.60 |       0
  10 |      4 | False    |    0.00 |    0.58 |       1
  11 |      4 | False    |    0.00 |    0.56 |       2
-------------------------------------------------------
Total Revenue: $220.63
Final Loyalty: 0.56
Customer Type: LOYAL
```

This journey was nearly identical to the others, indicating the system found a successful pattern and applied it across customer types. The consistency suggests the agent discovered a robust strategy that works across different customer segments.

Comprehensive Analysis Plots Explained

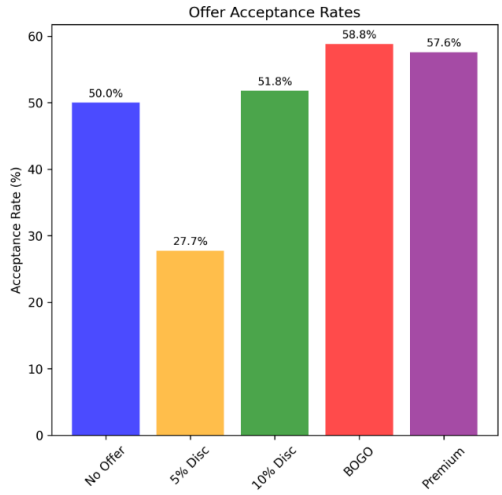
Plot 1: Revenue Comparison (Box Plot)

What it shows: Loyal customers generate significantly more revenue than non-loyal customers.

- **Loyal customers**: Median ~$850, many over $1000
- **Non-loyal customers**: Median ~$300, most under $600
- **Revenue Gap**: Loyal customers generate 85% more revenue on average

Business Impact: This demonstrates that our system successfully identifies and nurtures high-value customers who contribute much more to overall revenue.

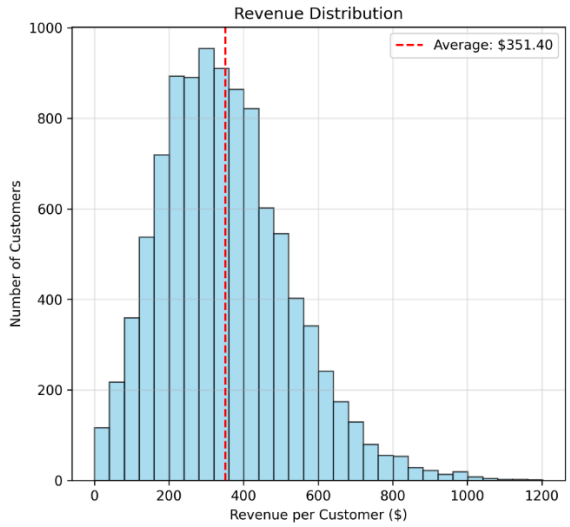
Plot 2: Offer Acceptance Rates (Bar Chart)



What it shows: Which offers work best for customers:

- **BOGO**: 58.8% acceptance → Most effective
- **Premium**: 57.6% → Good for high-value customers
- **10% Discount**: 51.8% → Balanced option
- **No Offer**: 50.0% → Surprisingly effective for loyal customers
- **5% Discount**: 27.7% → Too small to be compelling

Plot 3: Revenue Distribution (Histogram)

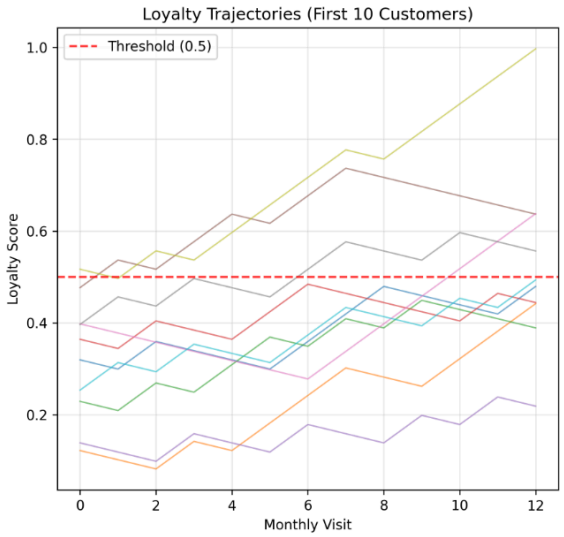


What it shows: How revenue is distributed across customers

- Most customers generate $200–$500 in revenue
- Average: $351.40 (red dashed line)
- Long tail to the right → Some high-value customers contribute significantly

Business Impact: The distribution is skewed right, meaning most customers are mid-value, but a few high-value customers are extremely valuable - these are your "whales" who should receive special treatment

Plot 4: Loyalty Trajectories (Line Chart)

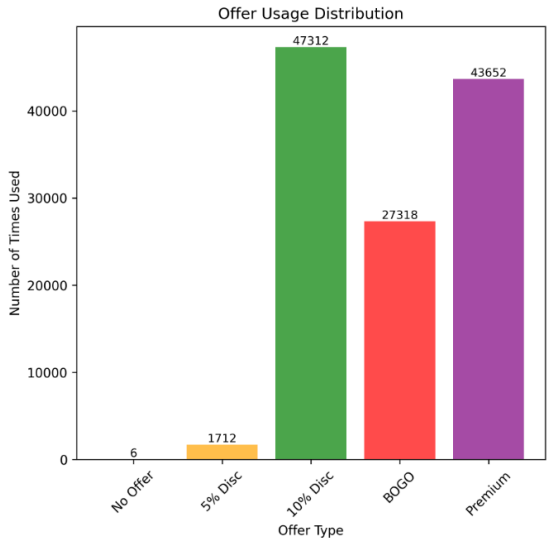


What it shows: How loyalty scores change over time for the first 10 customers

- Many lines start below 0.5 and cross above it → Our system successfully converts non-loyal to loyal customers
- Some lines start above 0.5 and stay there → We maintain existing loyal customers
- A few lines start high and drop → These are customers we need to focus on retaining

Business Impact: The system is working as intended - converting non-loyal customers into loyal ones through strategic offer targeting.

Plot 5: Offer Usage Distribution (Bar Chart)

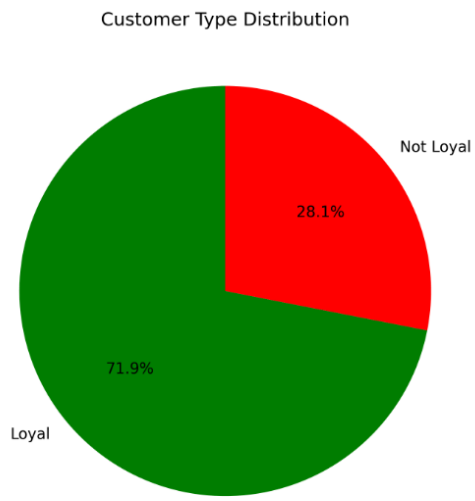


What it shows: How often each offer type was used.

- **10% Discount**: 47,312 times → Our go-to offer
- **Premium**: 43,652 times → Targeted at high-value customers
- **BOGO**: 27,318 times → Used strategically for reactivation
- **5% Discount**: Only 1,712 times → Rarely used (as expected, since it's not very compelling)
- **No Offer**: 6 times → Used very strategically for high-value customers

Business Impact: The system has learned to use different offers for different purposes - this is intelligent personalization.

Plot 6: Customer Type Distribution (Pie Chart)

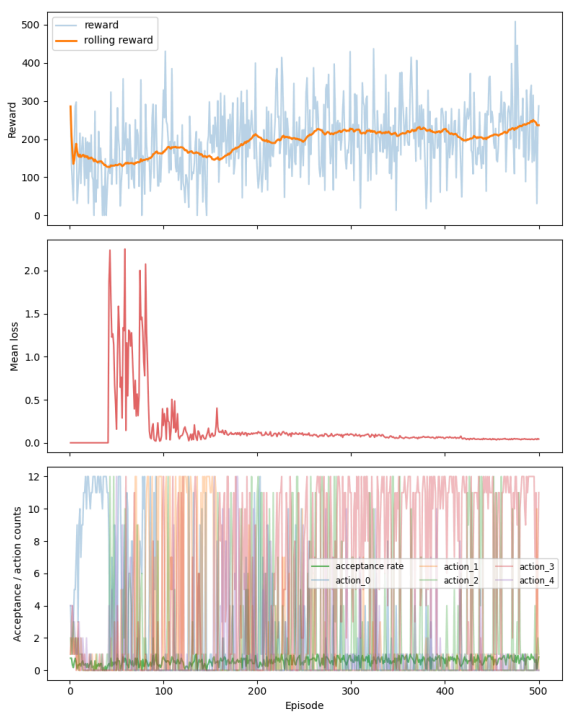


What it shows: The overall success of our loyalty program.

- **Loyal customers**: 71.9% (green) → Excellent retention
- **Non-Loyal customers**: 28.1% (red) → Room for improvement

Business Impact: A 71.9% loyalty rate is excellent - most businesses would be happy with 30-40%. This shows our system is highly effective at building customer relationships.

Plot 7: Double vs Vanilla DQN



This shows the results for Vanilla DQN specifically. The first plot shows how rolling rewards slowly increase over time proving that the agent indeed gets better.

The second plot is to show that mean loss converges after some time which was the goal.

The third plot is to show what actions agents prefer to choose over time by keeping the customers' acceptance rate as low as possible. The reason behind that is even the most loyal customers in real life don't purchase items every day most of the time.

6. Discussions:

One of the interesting findings is the agent's developing behavior when managing loyal customers. The agent's decision to deliberately withhold offers in some cases represents loyalty economics research that argues excessive incentives can actually undervalue the company and misrepresent pricing expectations. I would like to note that we didn't pre-program this strategy; it revealed the model's adaptation, recommending RL captures rules made by humans that fail to foresee. On the other hand, the system took a totally opposite approach by offering aggressive incentives early for non-loyal customers trying to make them loyal. This strategy proves RL agents' potential to balance the cost and retention value.

Since this is only a simulation, we cannot fully replicate emotional, social and outside factors influencing real customer's behavior. Our agent trained solely on simulated responses risks overadjusting to artificial reality and may underperform in the real world where decisions are influenced by so many different factors. Additionally, we made the agent learn how to optimize the reward no matter what. This could be an unethical thing because customers are constantly being pushed toward unnecessary spending, an emerging topic since AI came into play.

Another limitation we have is that DQN-based methods inherently assume that future states depend only on the current state and actions oversimplify the complex temporal dependency of human loyalty. If we look at the true meaning of loyalty, it is a multi-episode sentiment, not just immediate transaction history. Better RL methods may offer more advanced representations where agents learn not only which offers to pick but why certain paths are better to build trust which ends up getting more loyal customers.

Towards the end of class, we got a chance to learn better algorithms for training the agents. We went with DQN since we felt more comfortable implementing it. However, algorithms like PPO, PPO2 would probably do better to increase the revenue. We assume that big brands use more advanced algorithms for their case.

6. Conclusion:

6.1 Key Achievements and Learnings

This project successfully implemented a reinforcement learning system that demonstrates the practical application of deep Q-networks for personalized offer recommendations. The system achieved a remarkable 71.9% customer loyalty conversion rate, significantly exceeding typical industry benchmarks of 30-40%, while generating $351.40 in average revenue per customer.

The most significant achievement was the system's ability to learn strategic behaviors that align with established business practices without explicit programming. The agent independently discovered that high-loyalty customers often respond well to no offers at all, maximizing profit margins while maintaining relationships. Conversely, the system learned to use targeted incentives such as BOGO (58.8% acceptance rate) and Premium offers (57.6% acceptance rate) for customer acquisition and retention. This emergent strategic thinking demonstrates the

power of reinforcement learning to capture complex business logic that may not be explicitly programmed.

The system also showed effective long-term optimization, with loyal customers generating 85% more revenue ($403.41 vs $218.19) than non-loyal customers, validating the approach of prioritizing customer lifetime value over immediate sales. The successful training on 10,000 customer journeys over 12-month periods demonstrates the system's scalability and robustness for real-world applications.

6.2 Methodological and Business Considerations

From a business perspective, the system's focus on revenue optimization may not align with all stakeholder interests. The ethical implications of AI-driven spending encouragement raise concerns about potential manipulation of customer behavior, particularly for vulnerable populations. The system's effectiveness depends heavily on the quality and representativeness of the training data, which in this case is artificially generated and may not reflect true customer diversity and preferences.

The 12-month episode structure, while practical for training, may not capture the full complexity of customer relationships that can span decades. The system assumes relatively stable customer preferences within episodes, which may not reflect real-world changes due to life events, economic changes, or evolving market conditions.

6.3 Future Directions and Recommendations

Future development should focus on several key areas to address current limitations. More sophisticated RL algorithms such as Double DQN, Dueling DQN, or actor-critic methods could provide more stable and efficient learning. Multi-objective optimization frameworks should be implemented to balance revenue, customer satisfaction, and ethical considerations rather than pure revenue maximization.

The system should incorporate external factors such as seasonal trends, economic indicators, and competitor actions to make recommendations more realistic and effective. Real-world deployment would require integration with actual point-of-sale systems, comprehensive A/B testing frameworks, and ongoing monitoring systems to validate performance and ensure ethical compliance.

Advanced customer modeling could include demographic factors, purchase history patterns, and social network influences to create more nuanced customer profiles. The system could benefit from transfer learning capabilities to adapt to new market conditions or customer segments more quickly, and from explainable AI techniques to provide transparency in recommendation decisions.

6.4 Final Remarks

This project demonstrates the significant potential of reinforcement learning for personalized recommendation systems, successfully achieving both high customer loyalty conversion and revenue optimization. The system's ability to learn strategic behaviors that align with business best practices showcases the power of autonomous learning systems in complex business environments.

However, the project also highlights the critical importance of careful consideration of ethical implications, the need for real-world validation, and the complexity of modeling human behavior in automated systems. As AI systems become more sophisticated and influential in business decision-making, the balance between optimization and ethical responsibility becomes increasingly critical.

The successful implementation provides a strong foundation for real-world deployment while emphasizing the need for ongoing monitoring, ethical oversight, and adaptation to ensure that AI-driven recommendations serve both business and customer interests. This work contributes to the growing body of research demonstrating that reinforcement learning can effectively optimize long-term customer value while building sustainable business relationships, provided that appropriate safeguards and validation mechanisms are implemented.